CifアプリケーションのテンプレートになるServlet

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-054
画面名	CifアプリケーションのテンプレートになるServlet
会社名	
作成日	

概要

概要説明

CifアプリケーションのテンプレートになるServletの入力・表示・要求処理を行う。

情報	内容
対象種別	common

情報	内容
開始契機	呼出元からの要求
処理対象	DB2.ＵＤＢ管理ＤＢ

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

init

処理概要

Cifアプリケーションのテンプレートになる Servlet で「init」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理11：詳細仕様参照）。
2. 「対象項目」を設定する。

3. 関連情報を取得・設定する（処理12：詳細仕様参照）。

4. 関連情報を取得・設定する（処理13：詳細仕様参照）。

5. 関連情報を取得・設定する（処理14：詳細仕様参照）。

6. 関連情報を取得・設定する（処理15：詳細仕様参照）。

7. 共通処理呼出生成を実行する。

8. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

8.1. 通常処理：「条件16を満たす（原本条件は補足説明を参照）、かつ対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

8.1.1. 条件成立時：関連情報を取得・設定する（処理17：詳細仕様参照）。

8.2. 通常処理：「条件18を満たす（原本条件は補足説明を参照）、かつ対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

8.2.1. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

8-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

8-例外1.1. 例外発生時：関連情報を取得・設定する（処理20：詳細仕様参照）。

doPost

処理概要

Cifアプリケーションのテンプレートになる Servlet で「doPost」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

DB操作の詳細

DB操作 SQL-56387246D482

情報	内容
操作ID	SQL-56387246D482
対象テーブル	DB2.ＵＤＢ管理ＤＢ
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ＤＢ名 = 'Ｕ調査カード'
確定・取消	当該メソッド内で自動コミット設定を行う。例外時の処理順は詳細仕様の分岐に従う。
原本	n2framework/WEB-INF/src/jhkcommon/OjiServlet.java:244 / check@244

doGet

処理概要

CifアプリケーションのテンプレートになるServletで「doGet」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

DB操作の詳細

DB操作 SQL-56387246D482

情報	内容
操作ID	SQL-56387246D482
対象テーブル	DB2.ＵＤＢ管理ＤＢ
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ＤＢ名 = 'Ｕ調査カード'
確定・取消	当該メソッド内で自動コミット設定を行う。例外時の処理順は詳細仕様の分岐に従う。
原本	n2framework/WEB-INF/src/jhkcommon/OjiServlet.java:244 / check@244

ojiServletMain

処理概要

Cifアプリケーションのテンプレートになるservletで「ojiServletMain」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

1.2. 通常処理：関連情報を取得・設定する（処理13：詳細仕様参照）。

1.3. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

1.4. 通常処理：「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.4.1. 条件成立時：関連情報を取得・設定する（処理15：詳細仕様参照）。

1.5. 通常処理：「条件16を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.5.1. 条件成立時：入力値の制約と変換結果を確認する。

1.6. 通常処理：関連情報を取得・設定する（処理17：詳細仕様参照）。

1.7. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

1-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：関連情報を取得・設定する（処理19：詳細仕様参照）。

1-例外1.2. 例外発生時：「条件20を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1-例外1.2.1. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

1-例外1.2.2. 条件成立時：関連情報を取得・設定する（処理22：詳細仕様参照）。

DB操作の詳細

DB操作 SQL-56387246D482

情報	内容
操作ID	SQL-56387246D482
対象テーブル	DB2.ＵＤＢ管理ＤＢ

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ＤＢ名 = 'Ｕ調査カード'
確定・取消	当該メソッド内で自動コミット設定を行う。例外時の処理順は詳細仕様の分岐に従う。
原本	n2framework/WEB-INF/src/jhkcommon/OjiServlet.java:244 / check@244

startMain

処理概要

Cifアプリケーションのテンプレートになる Servlet で「startMain」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がある	処理の異常を通知し、この経路を終了する

exitMain

処理概要

CifアプリケーションのテンプレートになるServletで「exitMain」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連する処理へ制御を渡す。詳細な実行経路は原本対応を参照する。

errorControl

処理概要

CifアプリケーションのテンプレートになるServletで「errorControl」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目」を設定する。
2. 「対象項目」を設定する。
3. 「対象項目」を設定する。
4. 「対象項目」を設定する。
5. 関連情報を取得・設定する（処理22：詳細仕様参照）。
6. 「<FONT SIZE=3 COLOR=#0000A0><B>処理がエラーとなりました。</B></FONT>

\n」を設定する。
7. 関連情報を取得・設定する（処理23：詳細仕様参照）。
8. 「条件24を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

8.1. 条件成立時：「<tr><td width=100 valign=top>エラー理由＝</td>」を設定する。

8.2. 条件成立時：関連情報を取得・設定する（処理25：詳細仕様参照）。

8-別.1. 条件不成立時：「システムエラーが発生しています。システム部へ問い合わせてください。

\n\n」を設定する。

1. 「条件26を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.1. 条件成立時：関連情報を取得・設定する（処理27：詳細仕様参照）。

9.2. 条件成立時：関連情報を取得・設定する（処理28：詳細仕様参照）。

9.3. 条件成立時：関連情報を取得・設定する（処理29：詳細仕様参照）。

9.4. 条件成立時：「<input type=submit value=もう一度入力画面に戻る>\n」を設定する。

9.5. 条件成立時：関連情報を取得・設定する（処理30：詳細仕様参照）。

9-別.1. 条件不成立時：関連情報を取得・設定する（処理31：詳細仕様参照）。

9-別.2. 条件不成立時：関連情報を取得・設定する（処理32：詳細仕様参照）。

9-別.3. 条件不成立時：関連情報を取得・設定する（処理33：詳細仕様参照）。

9-別.4. 条件不成立時：「<input type=submit value=もう一度入力画面に戻る>\n」を設定する。

9-別.5. 条件不成立時：関連情報を取得・設定する（処理34：詳細仕様参照）。

1. 関連情報を取得・設定する（処理35：詳細仕様参照）。
2. 関連情報を取得・設定する（処理36：詳細仕様参照）。
3. 関連情報を取得・設定する（処理37：詳細仕様参照）。
4. 関連情報を取得・設定する（処理38：詳細仕様参照）。
5. 関連情報を取得・設定する（処理39：詳細仕様参照）。
6. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

15.1. 通常処理：関連情報を取得・設定する（処理40：詳細仕様参照）。

15.2. 通常処理：関連情報を取得・設定する（処理41：詳細仕様参照）。

15.3. 通常処理：関連情報を取得・設定する（処理42：詳細仕様参照）。

15.4. 通常処理：使用した接続・処理資源を解放する。

15-例外1. 例外条件「Exception eio」に該当した場合に次の手順を行う。型名は原本の表記。

makeResponseHeader

処理概要

CifアプリケーションのテンプレートになるServletで「makeResponseHeader」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目」を設定する。
2. 「対象項目」を設定する。
3. 「対象項目」を設定する。
4. 「対象項目」を設定する。

createConnection

処理概要

CifアプリケーションのテンプレートになるServletで「createConnection」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理5：詳細仕様参照）。

2.2. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

2.3. 通常処理：変更内容の確定を明示的に制御する。

2-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

make

処理概要

Cifアプリケーションのテンプレートになる Servlet で「make」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連する処理へ制御を渡す。詳細な実行経路は原本対応を参照する。

check

処理概要

Cifアプリケーションのテンプレートになる Servlet で「check」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理20：詳細仕様参照）。
2. 関連情報を取得・設定する（処理21：詳細仕様参照）。
3. 関連情報を取得・設定する（処理22：詳細仕様参照）。
4. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：関連情報を取得・設定する（処理23：詳細仕様参照）。

4.2. 通常処理：関連情報を取得・設定する（処理24：詳細仕様参照）。

4.3. 通常処理：変更内容の確定を明示的に制御する。

4.4. 通常処理：関連情報を取得・設定する（処理25：詳細仕様参照）。

4.5. 通常処理：関連情報を取得・設定する（処理26：詳細仕様参照）。

4.6. 通常処理：関連情報を取得・設定する（処理27：詳細仕様参照）。

4.7. 通常処理：「select 処理中ＦＬＧ」を設定する。

4.8. 通常処理：「 from DB2.ＵＤＢ管理ＤＢ」を設定する。

4.9. 通常処理：「 where ＤＢ名 = 'Ｕ調査カード'」を設定する。

4.10. 通常処理：関連情報を取得・設定する（処理28：詳細仕様参照）。

4.11. 通常処理：繰返し条件「条件29を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.11.1. 繰返しの各回：「rs.getLong("処理中ＦＬＧ") が等しい 0」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.11.1.1. 条件成立時：関連情報を取得・設定する（処理30：詳細仕様参照）。

4-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

4-例外1.1. 例外発生時：関連情報を取得・設定する（処理31：詳細仕様参照）。

4-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

4-後処理.1. 終了時の共通後処理：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4-後処理.1.1. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4-後処理.1.1.1. 条件成立時：使用した接続・処理資源を解放する。

4-後処理.1.2. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4-後処理.1.2.1. 条件成立時：使用した接続・処理資源を解放する。

4-後処理.1.3. 通常処理：関連情報を取得・設定する（処理32：詳細仕様参照）。

4-後処理.1-例外1. 例外条件「SQLException e1」に該当した場合に次の手順を行う。型名は原本の表記。

4-後処理.1-例外1.1. 例外発生時：関連情報を取得・設定する（処理33：詳細仕様参照）。

1. 「条件34を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-56387246D482

情報	内容
操作ID	SQL-56387246D482
対象テーブル	DB2.ＵＤＢ管理ＤＢ
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ＤＢ名 = 'Ｕ調査カード'
確定・取消	当該メソッド内で自動コミット設定を行う。例外時の処理順は詳細仕様の分岐に従う。
原本	n2framework/WEB-INF/src/jhkcommon/OjiServlet.java:244 / check@244

結果・例外時の扱い

到達条件	結果
条件35を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

setServletName

処理概要

Cifアプリケーションのテンプレートになる Servlet で「setServletName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setDatabaseType

処理概要

Cifアプリケーションのテンプレートになる Servlet で「setDatabaseType」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setOutputType

処理概要

Cifアプリケーションのテンプレートになる Servlet で「setOutputType」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setInPutHtmlName

処理概要

Cifアプリケーションのテンプレートになる Servlet で「setInPutHtmlName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setOutPutHtmlName

処理概要

Cifアプリケーションのテンプレートになる Servlet で「setOutPutHtmlName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setOutPutHtmlName2

処理概要

Cifアプリケーションのテンプレートになる Servlet で「setOutPutHtmlName2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setPdfFileName

処理概要

Cifアプリケーションのテンプレートになる Servlet で「setPdfFileName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setOutPutListName

処理概要

Cifアプリケーションのテンプレートになる Servlet で「setOutPutListName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setErrorReturnClassName

処理概要

CifアプリケーションのテンプレートになるServletで「setErrorReturnClassName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setErrorReturnHtmlName

処理概要

CifアプリケーションのテンプレートになるServletで「setErrorReturnHtmlName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

setNotUsed

処理概要

CifアプリケーションのテンプレートになるServletで「setNotUsed」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

getHtmlSkeleton

処理概要

CifアプリケーションのテンプレートになるServletで「getHtmlSkeleton」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getFilePath

処理概要

CifアプリケーションのテンプレートになるServletで「getFilePath」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getOutPutHtmlName

処理概要

CifアプリケーションのテンプレートになるServletで「getOutPutHtmlName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getInPutHtmlName

処理概要

CifアプリケーションのテンプレートになるServletで「getInPutHtmlName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getPdfFileName

処理概要

CifアプリケーションのテンプレートになるServletで「getPdfFileName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

get47Status

処理概要

Cifアプリケーションのテンプレートになるservletで「get47Status」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

DB関連処理

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
init	処理1	initException=e
init	処理2	htmlSkeleton2=cCom.CifComonHTMLRead(outPutHtmlName2,"")
init	条件3	!outPutHtmlName2.equals("")
init	処理4	htmlSkeleton=cCom.CifComonHTMLRead(outPutHtmlName,"")
init	条件5	!outPutHtmlName.equals("")
init	処理6	cCom = new CifComon(sFilePath, sDBServer, "", "")
init	処理7	sPath= scntxt.getInitParameter("SSERVLETPATH")
init	処理8	sFilePath= scntxt.getInitParameter("SFILEPATH")
init	処理9	ServletContext scntxt = this.getServletContext()

イベント	参照番号	原本条件・処理
init	処理10	super.init(config)
init	処理11	super.init(config)
init	処理12	ServletContext scntxt = this.getServletContext()
init	処理13	sFilePath= scntxt.getInitParameter("SFILEPATH")
init	処理14	sPath= scntxt.getInitParameter("SSERVLETPATH")
init	処理15	cCom = new CifComon(sFilePath, sDBServer, "", "")
init	条件16	!outPutHtmlName.equals("")
init	処理17	htmlSkeleton=cCom.CifComonHTMLRead(outPutHtmlName,"")
init	条件18	!outPutHtmlName2.equals("")
init	処理19	htmlSkeleton2=cCom.CifComonHTMLRead(outPutHtmlName2,"")
init	処理20	initException=e
doPost	処理1	ojiServletMain(req,res)
doPost	処理2	ojiServletMain(req,res)
doGet	処理1	ojiServletMain(req,res)
doGet	処理2	ojiServletMain(req,res)
ojiServletMain	処理1	this.getServletContext().log("log",e)
ojiServletMain	処理2	e.printStackTrace()
ojiServletMain	条件3	e.toString().startsWith("java.sql.SQLException")
ojiServletMain	処理4	errorControl(e,req,res)

イベント	参照番号	原本条件・処理
ojiServletMain	処理5	exitMain()
ojiServletMain	処理6	mainControl(req,res)
ojiServletMain	条件7	!new String(inputParams.getBytes("iso-8859-1"),"Windows-31J").equals("LoanMain.html")
ojiServletMain	処理8	inputParams = ""
ojiServletMain	処理9	inputParams=req.getParameter("HTML_NAME")
ojiServletMain	処理10	String inputParams
ojiServletMain	処理11	startMain()
ojiServletMain	処理12	startMain()
ojiServletMain	処理13	String inputParams
ojiServletMain	処理14	inputParams=req.getParameter("HTML_NAME")
ojiServletMain	処理15	inputParams = ""
ojiServletMain	条件16	!new String(inputParams.getBytes("iso-8859-1"),"Windows-31J").equals("LoanMain.html")
ojiServletMain	処理17	mainControl(req,res)
ojiServletMain	処理18	exitMain()
ojiServletMain	処理19	errorControl(e,req,res)
ojiServletMain	条件20	e.toString().startsWith("java.sql.SQLException")
ojiServletMain	処理21	e.printStackTrace()

イベント	参照番号	原本条件・処理
ojiServletMain	処理22	this.getServletContext().log("log",e)
errorControl	処理1	e.printStackTrace(pw)
errorControl	処理2	pw.println(errorHtml)
errorControl	処理3	PrintWriter pw = res.getWriter()
errorControl	処理4	errorHtml+="<tr><td>StackTrace</td><td>"+"\n"
errorControl	処理5	errorHtml+="<tr><td>e.toString</td><td>"+e.toString()+"</td></tr>\n"
errorControl	処理6	errorHtml+="<tr><td>e.getMessage</td><td>"+e.getMessage()+"</td></tr>\n"
errorControl	処理7	errorHtml+="<tr><td width=110>className</td><td>"+servletName+"</td></tr>\n"
errorControl	処理8	errorHtml+="<table width=630 border=1>\n"
errorControl	処理9	errorHtml+="</FORM>\n"
errorControl	処理10	errorHtml+="<input type=hidden name=SERVLET_LINE value=LineMakeBrno>\n"
errorControl	処理11	errorHtml+="<input type=hidden name=HTML_NAME value="+inPutHtmlName+">\n"
errorControl	処理12	errorHtml+="<FORM name=eroor method=post action=HtmlHandler>\n"
errorControl	処理13	errorHtml+="</FORM>\n"
errorControl	処理14	errorHtml+="<input type=hidden name=SERVLET_LINE value=LineMakeBrno>\n"

イベント	参照番号	原本条件・処理
errorControl	処理15	errorHtml+="<input type=hidden name=HTML_NAME value="+errorReturnHtmlName+">\n"
errorControl	処理16	errorHtml+="<FORM name=eroor method=post action="+errorReturnClassName+">\n"
errorControl	条件17	errorReturnClassName.equals("")
errorControl	処理18	errorHtml +="<td>"+CifErrorCode.getErrorMessage(e.getMessage())+"</td></tr></table> \n"
errorControl	条件19	e.toString().startsWith("jhkcommon.CifApplicationException")
errorControl	処理20	errorHtml+="<table width=630 border=0>\n"
errorControl	処理21	String errorHtml=""
errorControl	処理22	String errorHtml=""
errorControl	処理23	errorHtml+="<table width=630 border=0>\n"
errorControl	条件24	e.toString().startsWith("jhkcommon.CifApplicationException")
errorControl	処理25	errorHtml +="<td>"+CifErrorCode.getErrorMessage(e.getMessage())+"</td></tr></table> \n"
errorControl	条件26	errorReturnClassName.equals("")
errorControl	処理27	errorHtml+="<FORM name=eroor method=post action=HtmlHandler>\n"
errorControl	処理28	errorHtml+="<input type=hidden name=HTML_NAME value="+inPutHtmlName+">\n"

イベント	参照番号	原本条件・処理
errorControl	処理29	errorHtml+="<input type=hidden name=SERVLET_LINE value=LineMakeBrno>\n"
errorControl	処理30	errorHtml+="</FORM>\n"
errorControl	処理31	errorHtml+="<FORM name=eroor method=post action="+errorReturnClassName+">\n"
errorControl	処理32	errorHtml+="<input type=hidden name=HTML_NAME value="+errorReturnHtmlName+">\n"
errorControl	処理33	errorHtml+="<input type=hidden name=SERVLET_LINE value=LineMakeBrno>\n"
errorControl	処理34	errorHtml+="</FORM>\n"
errorControl	処理35	errorHtml+="<table width=630 border=1>\n"
errorControl	処理36	errorHtml+="<tr><td width=110>className</td><td>"+servletName+"</td></tr>\n"
errorControl	処理37	errorHtml+="<tr><td>e.getMessage</td><td>"+e.getMessage()+"</td></tr>\n"
errorControl	処理38	errorHtml+="<tr><td>e.toString</td><td>"+e.toString()+"</td></tr>\n"
errorControl	処理39	errorHtml+="<tr><td>StackTrace</td><td>"+"\n"
errorControl	処理40	PrintWriter pw = res.getWriter()
errorControl	処理41	pw.println(errorHtml)
errorControl	処理42	e.printStackTrace(pw)
createConnection	処理1	conn = ds.getConnection()

イベント	参照番号	原本条件・処理
createConnection	処理2	DataSource ds = (DataSource) new InitialContext().lookup(databaseType)
createConnection	処理3	Connection conn=null
createConnection	処理4	Connection conn=null
createConnection	処理5	DataSource ds = (DataSource) new InitialContext().lookup(databaseType)
createConnection	処理6	conn = ds.getConnection()
check	条件1	!flg
check	処理2	e1.printStackTrace()
check	処理3	con = null
check	処理4	e1.printStackTrace()
check	処理5	con = null
check	処理6	e1.printStackTrace()
check	処理7	con = null
check	処理8	e.printStackTrace()
check	条件9	rs.next()
check	処理10	flg = true
check	処理11	rs = stmt.executeQuery(sb.toString())
check	処理12	StringBuffer sb = new StringBuffer()
check	処理13	ResultSet rs = null

イベント	参照番号	原本条件・処理
check	処理14	stmt = con.createStatement()
check	処理15	con = ds.getConnection()
check	処理16	DataSource ds = (DataSource) new InitialContext().lookup("jdbc/Oracle_SP2NM")
check	処理17	boolean flg = false
check	処理18	Statement stmt = null
check	処理19	Connection con = null
check	処理20	Connection con = null
check	処理21	Statement stmt = null
check	処理22	boolean flg = false
check	処理23	DataSource ds = (DataSource) new InitialContext().lookup("jdbc/Oracle_SP2NM")
check	処理24	con = ds.getConnection()
check	処理25	stmt = con.createStatement()
check	処理26	ResultSet rs = null
check	処理27	StringBuffer sb = new StringBuffer()
check	処理28	rs = stmt.executeQuery(sb.toString())
check	条件29	rs.next()
check	処理30	flg = true
check	処理31	e.printStackTrace()

イベント	参照番号	原本条件・処理
check	処理32	con = null
check	処理33	e1.printStackTrace()
check	条件34	!flg
check	条件35	!flg
setServletName	処理1	this.servletName=servletName
setServletName	処理2	this.servletName=servletName
setDatabaseType	処理1	this.databaseType=databaseType
setDatabaseType	処理2	this.databaseType=databaseType
setOutputType	処理1	this.outputType=outputType
setOutputType	処理2	this.outputType=outputType
setInPutHtmlName	処理1	this.inPutHtmlName=inPutHtmlName
setInPutHtmlName	処理2	this.inPutHtmlName=inPutHtmlName
setOutPutHtmlName	処理1	this.outPutHtmlName=outPutHtmlName
setOutPutHtmlName	処理2	this.outPutHtmlName=outPutHtmlName
setOutPutHtmlName2	処理1	this.outPutHtmlName2=outPutHtmlName2
setOutPutHtmlName2	処理2	this.outPutHtmlName2=outPutHtmlName2
setPdfFileName	処理1	this.pdfFileName=pdfFileName
setPdfFileName	処理2	this.pdfFileName=pdfFileName
setOutPutListName	処理1	this.outPutListName=outPutListName

イベント	参照番号	原本条件・処理
setOutPutListName	処理2	this.outPutListName=outPutListName
setErrorReturnClassName	処理1	this.errorReturnClassName=errorReturnClassName
setErrorReturnClassName	処理2	this.errorReturnClassName=errorReturnClassName
setErrorReturnHtmlName	処理1	this.errorReturnHtmlName=errorReturnHtmlName
setErrorReturnHtmlName	処理2	this.errorReturnHtmlName=errorReturnHtmlName
setNotUsed	処理1	this.notUsed=notUsed
setNotUsed	処理2	this.notUsed=notUsed